

AI-Powered Farming Assistant for Kerala Farmers

Comprehensive Technical Project Report

Frontend, backend, AI services, data flow, deployment, testing, and RAG roadmap

Prepared from the current repository implementation

Technology status documented as implemented, not only planned

1. Executive summary

This project is a full-stack agricultural decision-support application focused on Kerala farmers. It provides disease detection from crop-leaf images, disease-treatment guidance, crop suitability recommendations, weather-based farming advice, market intelligence, and a conversational AI assistant. The user interface is a React single-page application; the service layer is a FastAPI application; model and AI integrations use PyTorch/Torchvision, Groq, WeatherAPI, and Kerala market data.

The application is designed so each major page calls a backend API and renders its structured response. The frontend has loading, error, and result states for these requests.

Key user outcomes

- Upload a leaf image and receive a disease class and confidence score for paddy, pepper, rubber, coconut, or banana.
- Receive chemical, organic, dosage, prevention, symptom, and precaution information for a detected disease.
- Choose a location, soil type, and irrigation method to receive ranked crop recommendations.
- Use coordinates to obtain current conditions and operational weather recommendations.
- Compare market price signals and selling guidance by crop and Kerala district.
- Ask farming questions through a Groq-powered conversational endpoint with recent-message context.

Important implementation note

The repository contains a `knowledge_base` folder with agricultural PDFs, but the files intended to implement retrieval-augmented generation (`rag.py`, `vector_store.py`, `embeddings.py`, `prompts.py`, and `chatbot.py`) are empty. Therefore, the currently running assistant is a Groq chat-completions assistant with conversation history, not a fully implemented RAG pipeline. Section 9 specifies the RAG architecture needed to make the assistant genuinely retrieval augmented.

2. System architecture

The application follows a browser-to-API architecture. The frontend owns display and interaction state, while the backend performs model inference, calls external services, and returns JSON contracts.

Browser (React/Vite) -> Axios API client -> FastAPI routes -> ML models / Groq / WeatherAPI / Kerala market source -> JSON response -> React components

Layer	Responsibility	Current technology
Presentation	Pages, language switching, form controls, charts, animations, result cards	React 19, TypeScript, Vite, React Router, i18next, Framer Motion, Recharts, Leaflet
Client data	HTTP requests, API error translation, loading/mutation state	Axios, TanStack React Query
API	Validation, routing, CORS, service orchestration, JSON responses	Python, FastAPI, Pydantic, Uvicorn
Disease AI	Image classification and confidence calculation	PyTorch, Torchvision, timm EfficientNet-B0
Generative AI	Treatment, crop, weather, market, and chat language generation	Groq Python SDK; llama-3.3-70b-versatile

Layer	Responsibility	Current technology
External data	Forecast conditions and market records	WeatherAPI; Kerala Ecostat endpoint
Knowledge assets	Agronomy manuals in PDF form	Local PDF knowledge_base; RAG indexing not yet implemented

3. Frontend

The frontend is located in frontend/. It is a Vite-built TypeScript application with client-side routing. A shared Navbar and PageTransition wrap the pages; QueryProvider configures React Query.

Route	Page behavior	Backend endpoint
/	Home/landing page and navigation	No feature request required
/disease	Crop selector, image upload/camera, preview, result confidence ring	POST /api/disease/predict
/treatment	Tabbed symptoms, chemical, organic, dosage, prevention, precautions	GET /api/treatment
/crop	Location, soil, irrigation and ranked crop cards	POST /api/crop-advisor
/weather	Weather metrics, operational decisions, alerts, outlook	POST /api/weather
/market	Market score, price table, prediction chart, rankings, risks	POST /market/market
/assistant	Conversation interface, quick prompts, typing state	POST /api/assistant/chat

Client communication

- src/api/client.ts creates a 60-second Axios client. API errors are converted to readable Error messages from FastAPI detail fields.
- src/api/endpoints.ts centralizes request creation. Disease upload uses multipart/form-data; other modules use JSON; treatment uses query parameters.
- The default VITE_API_BASE_URL is http://localhost:8000. Vite development proxies /api and /market to the backend.
- Mutation and query states render SkeletonCard placeholders while requests are running and ErrorCard on failures.
- The UI supports English, Malayalam, Hindi, and Tamil JSON locale files; not every dynamic AI response is translated because those responses originate from the backend.

Visualization and UX libraries

- Framer Motion animates routes, cards, tabs, confidence rings, and message appearance.
- Recharts draws the market price-prediction line chart.
- React Leaflet and the location picker support geographic selection; the picker also uses Nominatim search.
- Lucide React supplies interface icons. CSS uses design tokens and reusable components in src/index.css.

4. Backend and API contracts

The FastAPI entry point is backend/app.py. It enables CORS for HTTP browser origins, exposes / and /health, and registers feature routers. Pydantic schemas validate request bodies for weather, crop advisory, market, and chat requests.

Endpoint	Input	Core response consumed by UI
POST /api/disease/predict	multipart: crop, image	crop, prediction, confidence (0-100)
GET /api/treatment	crop and disease query values, optional	overview plus six arrays: symptoms, treatments, dosage, prevention, precautions
POST /api/crop-advisor	latitude, longitude, soil_type, irrigation	location, summary, best_crop, recommended_crops, not_recommended
POST /api/weather	latitude, longitude, crop	weather details and structured farming advice
POST /market/market	crop, district	prices, scorecards, trend prediction, decision, risk, alerts
POST /api/assistant/chat	message and optional conversation_history	reply

Backend service behavior

- disease_api stores the latest crop/disease prediction in in-process prediction_state so the treatment route can use it when explicit parameters are absent.
- weather_service fetches a 3-day WeatherAPI forecast, normalizes field names used by the UI, and enforces a network timeout.
- crop_advisor_api obtains weather and season data, asks Groq for crop recommendations, and normalizes generated field names such as crop/name and recommendation_rank/rank for the frontend.
- market_service fetches Kerala Ecostat records, filters them by crop, calculates district comparisons, price-based scores, windows, predictions, and uses Groq for market commentary. It has an estimated-price fallback when no matching commodity exists.
- groq_services provides safe JSON parsing plus fallback structured answers for treatment, weather, crop, and market generation when Groq fails.
- assistant_api now delegates to get_chat_response. It passes up to the latest ten valid conversation messages and returns a stable reply field. A provider failure returns a helpful message rather than an unhandled backend error.

5. Disease detection AI

Disease detection is the locally executed predictive model. The application provides one model flow per supported crop: banana, coconut, paddy, pepper, and rubber. Saved best_model.pth files are present under saved/disease//.

Inference flow

- The API receives an uploaded image and stores it in a temporary file.
- DiseasePredictor selects the crop dataset/classes and loads the crop-specific best checkpoint.
- The image is converted to RGB, resized to 224 x 224, converted to a tensor, and normalized with ImageNet mean and standard-deviation values.

- An EfficientNet-B0 backbone created through timm produces class logits. The final classifier is a dropout (0.3) plus linear layer sized to the number of classes.
- Softmax converts logits to probabilities. The highest probability becomes prediction and the confidence is returned as a percentage rounded to two decimals.
- The temporary uploaded file is removed in a finally block after inference.

Training and evaluation

- The project uses ImageFolder datasets with train, validation, and test splits, DataLoader batches of 32, a seeded configuration, and CUDA when available.
- Training configuration sets 10 epochs, learning rate 1e-4, and weight decay 1e-4.
- Evaluation calculates accuracy, weighted precision, weighted recall, and weighted F1; it writes classification reports and confusion-matrix images in backend/models/disease/reports/.
- Reported model metrics should be taken from the generated per-crop reports for a particular dataset/version; this report does not invent aggregate accuracy values.

6. Generative AI and assistant behavior

Groq is used for natural-language generation in five domain modules: treatment recommendation, crop advisory, weather advice, market insight, and chat. The backend requests structured JSON where the frontend expects fields, then parses or normalizes that output.

Current assistant flow

- The frontend appends the user's message to local state and sends the latest ten messages with the request.
- The backend adds a system prompt that directs the model to give concise, practical Kerala-farming guidance and to acknowledge uncertainty for weather, market, and pesticide questions.
- The latest user question is guaranteed to be added even if the client history is incomplete.
- Groq llama-3.3-70b-versatile generates the reply. The backend sends { reply: string } to the chat interface.
- This approach has conversational memory within the browser-provided recent history, but it does not retrieve source passages from the PDF knowledge base.

Safety and quality

- Prompts ask for practical advice and safety/label guidance, but generated agricultural advice must still be reviewed against local recommendations, product labels, and extension services.
- The assistant should not be presented as a substitute for a certified agronomist, pesticide label, or emergency authority.
- Production systems should add prompt-injection defense, input/output moderation, citations, rate limiting, request logging with privacy controls, and human escalation paths.

7. Weather and market intelligence

Weather advisory

- WeatherAPI forecast data is queried with latitude and longitude. The service returns location, temperatures, humidity, rain chance, precipitation, wind, cloud, UV, pressure, visibility, sunrise, and sunset.
- The weather prompt asks Groq for a summary, three action-plan steps, irrigation/spraying/harvesting boolean decisions with reasons, alerts, a three-day outlook, and Low/Medium/High risk.
- If Groq generation fails after weather is obtained, a deterministic fallback returns the same response shape. High rain probability causes conservative spray, irrigation, and harvest recommendations.
- If the weather provider itself is unavailable, the API returns a readable failure and the frontend displays its error card. In production, cache the last successful result and offer a clearly labelled stale-data fallback.

Market intelligence

- MarketService requests a Kerala Ecostat dataset and filters records using the selected crop. It includes token matching and estimated values for unmatched crop names.
- It calculates price range, district-market comparisons, price strength, demand/supply proxies, profitability, risk, market health, a selling window, and simple near-term price projections.
- Groq is used for the descriptive market insight and decision explanation. The UI renders both numerical tables/charts and this AI narrative.
- The UI currently displays price values with a rupee/kg label, while the source response also includes a unit field. Before production, normalize every source unit and display the actual unit explicitly to avoid misleading comparisons.

8. Data, persistence, and security

Current data assets

- Disease model checkpoints and evaluation reports are stored locally in the repository workspace.
- Agronomy PDFs are stored under backend/models/chatbot/knowledge_base, grouped by topics such as fertilizer and crop manuals.
- The current API state for a last disease prediction is process-memory only. It is not suitable for multi-user production because users can overwrite one another's latest prediction.
- Database package files exist, but the inspected database source does not currently provide a working persistence path for user accounts, conversations, predictions, or audit events.

Security and production recommendations

- Keep WEATHER_API_KEY and GROQ_API_KEY only in environment variables or a secret manager; never commit them or expose them to Vite/browser code.
- Replace broad CORS policy with specific deployed frontend origins.
- Add file size limits, MIME/content validation, image decompression protections, and authentication/authorization for uploads.
- Replace global prediction_state with request/session IDs and a database-backed prediction record.
- Add HTTPS, rate limits, structured logs without secrets, health/readiness checks, and dependency vulnerability updates.

9. RAG implementation roadmap

A Retrieval-Augmented Generation assistant grounds answers in trusted documents. It retrieves relevant sections from the local agricultural manuals before calling the LLM, then asks the LLM to answer only from those sections and return citations.

Recommended RAG pipeline

PDF manuals -> text extraction -> cleaned chunks with metadata -> embeddings -> FAISS/vector index -> similarity retrieval -> prompt with sources -> Groq answer with citations

Component	Recommended implementation	Repository target
Document loader	Use PyPDFLoader/pypdf; retain source filename, page, crop, topic, language	models/chatbot/rag.py
Chunking	Recursive chunking around 700-1,000 tokens with 100-150 overlap	models/chatbot/rag.py
Embeddings	sentence-transformers model such as all-MiniLM-L6-v2; normalize vectors	models/chatbot/embeddings.py
Vector store	FAISS local index persisted with document metadata	models/chatbot/vector_store.py
Prompting	System policy plus question and top 3-5 evidence chunks; demand citations	models/chatbot/prompts.py
Chat orchestration	Retrieve, optionally rerank, invoke Groq, return reply plus sources	models/chatbot/chatbot.py
API/UI	Extend ChatResponse with sources: [{title, page, excerpt}] and render citations	api/assistant_api.py; frontend assistant page

RAG acceptance criteria

- A question about a manual retrieves relevant chunks from that manual and includes visible source names/pages.
- When sources are insufficient, the assistant says so rather than fabricating a citation.
- A re-index command can rebuild the vector store after PDFs change.
- Tests cover retrieval relevance, source citation format, and answer behavior with no retrieved context.
- The frontend clearly labels cited manual advice versus general model guidance.

10. Setup, execution, and testing

Local setup

1. Create/activate .venv and install backend/requirements.txt. 2. Configure required provider keys in backend/.env.
3. Install frontend dependencies. 4. Run npm.cmd run dev from the project root, or use start.ps1. The backend runs at http://localhost:8000 and Vite runs at http://localhost:5173.

Common commands

```
npm.cmd --prefix frontend run build
.\.venv\Scripts\uvicorn backend.app:app --reload --host 0.0.0.0 --port 8000
npm.cmd run dev
```

Verification performed for this report

- The frontend production build completed successfully with TypeScript checking and Vite bundling.

- API contract smoke checks verified the response shapes consumed by the frontend for disease, treatment, weather, crop advisory, market intelligence, and chat using controlled service doubles.
- Python syntax compilation succeeded for the updated chat API and Groq service.
- A live WeatherAPI request could not be executed from the restricted report-generation environment; this is an environment network limitation, not a frontend contract result.

Known improvements

- Implement the RAG modules and citations described in Section 9.
- Add automated pytest API tests and React component/end-to-end tests to version control.
- Use durable, per-user storage instead of global prediction state.
- Add schema validation/normalization to every generated AI response before rendering.
- Code-split the frontend: the production build reports a JavaScript bundle larger than the default 500 kB warning threshold.
- Add accessibility testing, mobile performance budgets, offline/error recovery, and observability before deployment.

11. Conclusion

The project is a functional full-stack farming-assistance platform with a polished multi-module frontend, FastAPI integration layer, local image-classification models, and AI-assisted advisory features. Its strongest implementation areas are the disease workflow, structured feature APIs, and frontend result presentation. The next major milestone is converting the knowledge-base assets into a real RAG system with retrieval, citations, and durable data management. Completing that work will make the AI assistant verifiable, more trustworthy, and better suited to agricultural guidance.